

-- Exercício 1--

--a)

CREATE OR REPLACE FUNCTION bloquear_exclusao()

RETURNS TRIGGER

AS \$\$

BEGIN

IF OLD.quantidade > 0 THEN

RAISE EXCEPTION 'Não é possível remover livros com exemplares disponíveis no estoque.';

END IF;

RETURN OLD;

END;

\$\$ LANGUAGE plpgsql;

--b)

CREATE TRIGGER trg_bloquear_exclusao

BEFORE DELETE ON livro

FOR EACH ROW

EXECUTE FUNCTION bloquear_exclusao();

-- Exercício 2--

--a)

CREATE OR REPLACE FUNCTION log_exclusao_livro()

RETURNS TRIGGER

AS \$\$

BEGIN

```
INSERT INTO log_livro (titulo, quantidade_antiga, quantidade_nova, data_log)
VALUES (
    OLD.titulo,
    OLD.quantidade,
    NULL,
    CURRENT_TIMESTAMP
);

RETURN OLD;

END;

$$ LANGUAGE plpgsql;
```

```
--b)

CREATE TRIGGER trg_log_exclusao
AFTER DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();
```

-- Exercício 3--

```
--a)

CREATE OR REPLACE FUNCTION validar_limite_estoque()
RETURNS TRIGGER
AS $$
BEGIN

    IF NEW.quantidade > 100 THEN
```

```
RAISE EXCEPTION 'Quantidade de estoque inválida. O limite máximo permitido é de
100 exemplares.';
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
--b)
```

```
CREATE TRIGGER trg_validar_limite
```

```
BEFORE UPDATE ON livro
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION validar_limite_estoque();
```

```
-- Exercício 4 --
```

--a) BEFORE (Antes): Funciona como um segurança de festa que barra quem está de bermuda antes de entrar no evento. Ela age antes do dado ser salvo. AFTER (Depois): Funciona como um fotógrafo de festa que tira foto de quem já entrou. Ela age depois que o dado já foi salvo.

--b) Deve ser usada a trigger BEFORE. Por que , se alguém tentar cadastrar algo errado (como um estoque negativo), você precisa barrar esse erro antes que ele entre e estrague seu banco de dados.

--c) Deve ser usada a trigger AFTER. Por que para tirar o extrato ou histórico real (log), você precisa ter certeza de que a ação realmente aconteceu. Não faz sentido anotar no histórico algo que acabou sendo cancelado ou barrado.

--d) Imagine um sensor de presença de uma porta automática: Primeiro, o sensor valida se tem espaço livre (BEFORE). Se tiver, a porta abre e a pessoa entra. Só depois que ela entrou, o sistema soma mais um no contador de visitas (AFTER). Se você invertesse a ordem, o sistema contaria uma visita mesmo se a pessoa batesse a cara na porta fechada. No banco de dados é igual: fazer as coisas na ordem certa evita registrar mentiras e economiza o trabalho do sistema.

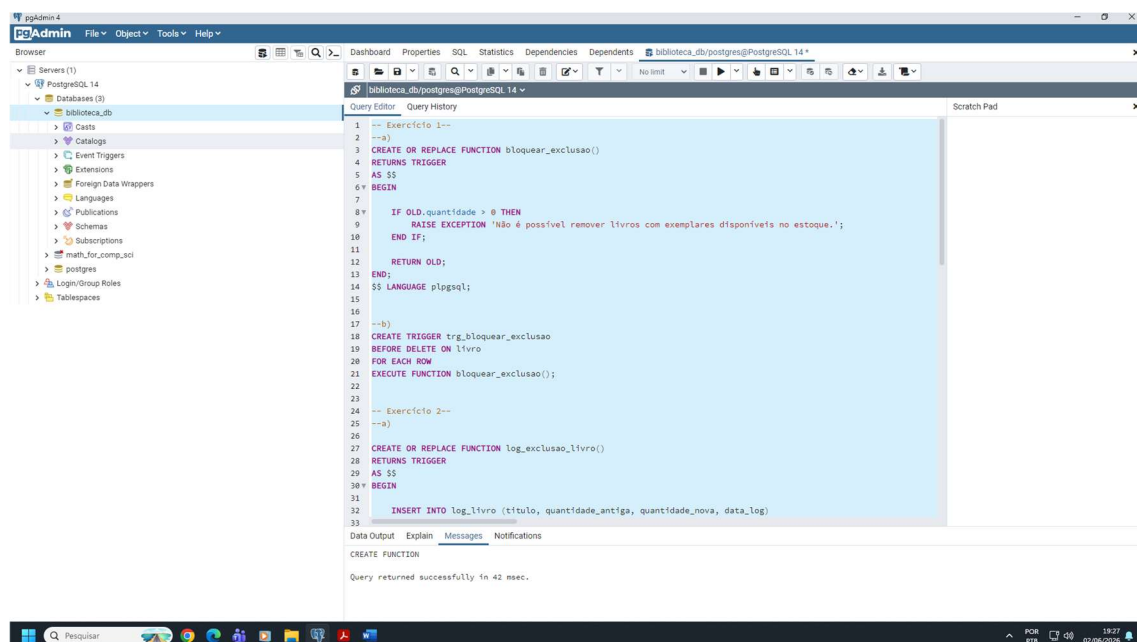
```
-- Exercício 5 --
```

--a) O maior risco é a perda de segurança e a corrupção dos dados, pois se o React, o Node ou o aplicativo falharem, ou se um funcionário mexer no banco de dados manualmente, o sistema aceitará dados errados e sem controle.

--b) A vantagem é a centralização da regra de segurança diretamente na base de dados, garantindo que qualquer aplicativo ou sistema conectado seja obrigado a seguir o mesmo padrão e as mesmas restrições.

--c) Elas ajudam funcionando como sensores automáticos e invisíveis que barram dados inválidos (como estoques negativos) e atualizam tabelas dependentes imediatamente, servindo como uma última barreira de proteção intransponível.

--d) Um exemplo real é o controle de estoque em um e-commerce, onde uma trigger diminui automaticamente a quantidade da mercadoria e gera um histórico de auditoria assim que a venda é confirmada.



The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show a PostgreSQL 14 server named 'biblioteca_db'. The 'Catalogs' section is expanded, showing 'Event Triggers' and 'Functions'. The 'Event Triggers' section is selected, showing a trigger named 'bloquear_exclusao'. The 'Functions' section is also expanded, showing a function named 'log_exclusao_livro'. The main pane displays the SQL code for the trigger and function. The trigger is a BEFORE DELETE trigger on the 'livro' table, and the function is a PL/pgSQL function that inserts a log entry into the 'log_livro' table.

```
1 -- Exercício 1--
2 --a)
3 CREATE OR REPLACE FUNCTION bloquear_exclusao()
4 RETURNS TRIGGER
5 AS $$
6 BEGIN
7
8 IF OLD.quantidade > 0 THEN
9 RAISE EXCEPTION 'Não é possível remover livros com exemplares disponíveis no estoque.';
10 END IF;
11
12 RETURN OLD;
13 END;
14 $$ LANGUAGE plpgsql;
15
16
17 --b)
18 CREATE TRIGGER trg_bloquear_exclusao
19 BEFORE DELETE ON livro
20 FOR EACH ROW
21 EXECUTE FUNCTION bloquear_exclusao();
22
23
24 -- Exercício 2--
25 --a)
26
27 CREATE OR REPLACE FUNCTION log_exclusao_livro()
28 RETURNS TRIGGER
29 AS $$
30 BEGIN
31 INSERT INTO log_livro (titulo, quantidade_antiga, quantidade_nova, data_log)
32 VALUES (OLD.titulo, OLD.quantidade, OLD.quantidade - 1, NOW());
33 END;
34 $$ LANGUAGE plpgsql;
```

Query returned successfully in 42 msec.